



Embedded and IOT System

Name: M.SAIM MUSTAFA

Class: BSAI 5TH

Reg #: 23-NTU-CS-1271

Course: Embedded and IOT System

Professor: MR. Nasir Mehmood.

Assignment # Home work -After mid

Question # 1 ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1. What is the purpose of `WebServer server(80);` and what does port 80 represent?

`WebServer server(80)` is used to create a web server on the device so it can handle HTTP requests from a browser.

Port 80 represents the default HTTP port.

2. Explain the role of `server.on("/", handleRoot);` in this program.

`server.on("/", handleRoot);` is used to define what the server should do when a client accesses the root URL (/)

3. Why is `server.handleClient();` placed inside the `loop()` function? What will happen if it is removed?

It is placed inside the `loop()` function so the server can continuously check for and process incoming client requests.

If it is removed, the server will not handle any client requests.

4. In `handleRoot()`, explain the statement:

`server.send(200, "text/html", html);`

The statement is used to send a response back to the client.

200 is the HTTP status code

“text/html” tells the browser that the content being sent is an HTML web page

Html is the actual data that will be displayed in the browser

5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside handleRoot()?

Displaying the last measured sensor values means the web page shows data that was already read earlier and stored in variables.

Taking a fresh DHT reading inside handleRoot() means the sensor is read every time the web page is requested.

Part B: Long Question

Describe the complete working of the ESP32 webserver-based temperature and humidity monitoring system.

When the ESP32 is powered on, it first connects to the Wi-Fi network using the given SSID and password. The program keeps checking the connection status until Wi-Fi is successfully connected. Once connected, the router assigns an IP address to the ESP32. This IP address is printed on the Serial Monitor and also shown on the OLED display.

After Wi-Fi connection, the web server is initialized on port 80. which allows the ESP32 to act like a normal website. The server is configured so that when the root URL (/) is opened in a browser, the handleRoot() function is executed. The server keeps listening for browser requests inside the loop() function using server.handleClient().

A push button is used to control when the sensor is read. The button is configured using INPUT_PULLUP, so it normally stays high and goes low when pressed. When a valid button press is detected, the ESP32 reads temperature and humidity values from the DHT22 sensor.

These values are stored in variables and then displayed on the OLED screen. This method avoids continuous sensor reading and gives more stable results.

The same stored sensor values are also used for the web page.

When a user opens the ESP32 IP address in a browser, the `handleRoot()` function generates an HTML page dynamically. If no sensor reading is available, a message is shown asking the user to press the button. If readings exist, the temperature and humidity are displayed on the page.

The webpage includes a meta refresh tag that refreshes the page every 5 seconds.

Common problems and solutions

Sometimes the web page does not load. This usually happens when `server.handleClient()` is missing from the `loop()` function.

If the ESP32 fails to connect to Wi-Fi, the issue is usually incorrect credentials or a weak signal. DHT sensors may return invalid (NaN) values if they are read too frequently, which is why button-based reading is used.

If the OLED does not show anything, the problem is often a wrong I2C address or wiring issue

Question-2 Blynk Cloud Interfacing (blynk.cpp)

Part-A: Short Questions

1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?

The Blynk Template ID role is a unique identity of project in the blynk iot cloud. It tells the esp32 which template to use for widgets, data pins, and device settings.

It must match the cloud template exactly because if it doesn't, the esp32 won't know which template to connect to.

2. Differentiate between Blynk Template ID and Blynk Auth Token.

The Blynk Template ID identifies the project template in the Blynk cloud.

The Blynk Auth Token identifies the specific device created from that template

3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.

Using DHT22 code with a DHT11 sensor gives wrong readings because the timing and data format between the two sensors are different

DHT22 can measure temperature from -40C to 80C and humidity from 0–100% with 0.1C/0.1% precision, whereas DHT11 measures 0–50C and 20–90% humidity with only 1C/1% precision.

4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?

They are software-defined pins used to send or receive data between the ESP32 and the Blynk app

They are preferred over physical pins for cloud communication because they allow flexible control of widgets, support any type of data, and don't depend on the hardware layout.

5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?

It is used to run tasks repeatedly without stopping the program. Unlike `delay()`, which pauses everything

Part-B: Long Question

Explain the complete workflow of interfacing ESP32 with Blynk Cloud to display temperature

1. Creation of Blynk Template and Datastreams

First, a Blynk Template is created in the Blynk IoT cloud. This template defines the structure of the project: the widgets and the Virtual Pins associated with each data type.

Datastreams are added inside the template to store data from the device. For this project, one datastream is for temperature (V0) and another for humidity (V1).

These datastreams allow the ESP32 to send data to the cloud in a structured way, which the Blynk app can then display on widgets.

2. Role of Template ID, Template Name, and Auth Token

Template ID identifies the exact template in the Blynk cloud. The ESP32 must use this same ID to match the cloud setup otherwise, widgets won't work properly.

Template is the human readable name for the template. It helps organize projects in the cloud but does not affect functionality.

Auth Token uniquely identifies the device created from the template.

3. Sensor Configuration Issues (DHT11 vs DHT22)

The code uses DHT. Using the wrong sensor type causes incorrect readings because DHT11 and DHT22 have different timing and resolution.

Key difference: DHT22 measures -40C to 80C with 0.1C precision, while DHT11 measures 0–50C with 1C precision.

4. Sending Data Using Blynk.virtualWrite()

Sensor values are read using `dht.readTemperature()` and `dht.readHumidity()`.

These values are then sent to the cloud using Virtual Pins:

```
Blynk.virtualWrite(V0, t)
```

```
Blynk.virtualWrite(V1, h
```

The Virtual Pins correspond to the datastreams in the Blynk template, allowing real-time updates in the app.

The BlynkTimer is used to periodically send readings without blocking the program

5. Button and OLED Integration

physical button is connected to the ESP32 with `INPUT_PULLUP`.

Pressing the button triggers a manual sensor reading and updates both the OLED and Blynk cloud.

The OLED provides local feedback, while the Blynk app allows remote monitoring.

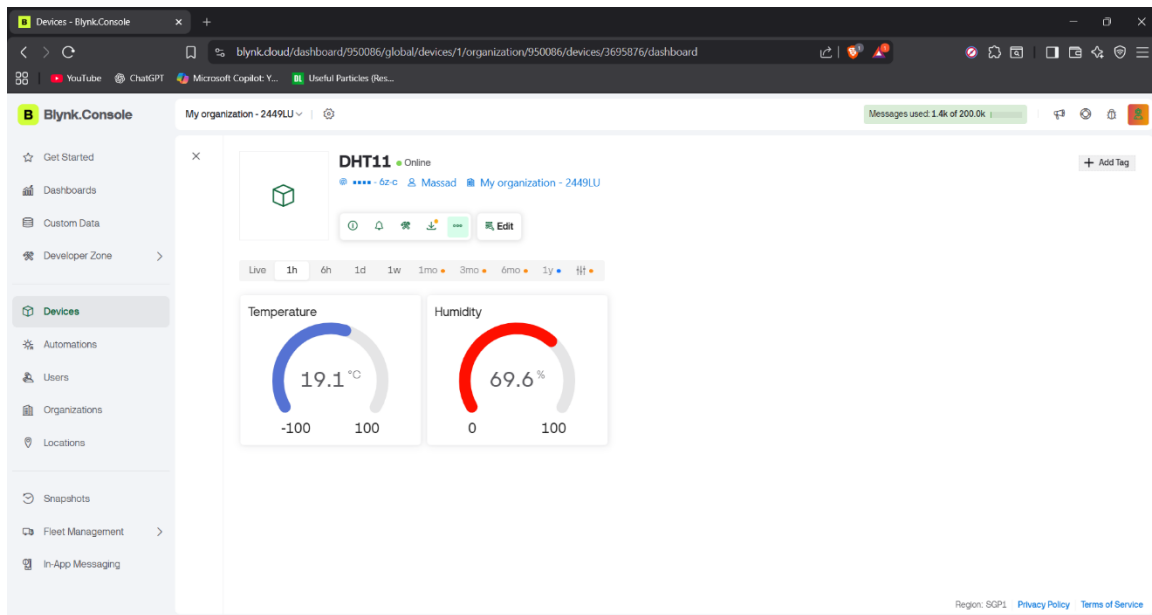
6. Common Problems and Solutions

Webpage/app shows wrong values or nothing Check Template ID and Auth Token they must match the cloud setup.

Sensor gives NaN readings Ensure DHTTYPE matches the sensor, and avoid reading too frequently.

ESP32 not connecting to Wi-Fi Verify SSID and password. Make sure Wi-Fi signal is stable.

Cloud updates delayed or frozen Use BlynkTimer instead of `delay()`,



Task B

Code:

Output:

